

微處理器系統個人實驗報告 - Lab 08

實驗主題：簡易中央處理器實作 (Simple CPU Implementation)

班級：資工二

學號：113590017 113590051

姓名：黃楷程 楊承諭

組別：第 19 組

組員：楊承諭、黃楷程

實驗日期：2026年6月3日

一、實驗內容與硬體規格

本次實驗的核心目標是設計並實作一個支援基礎指令集架構 (ISA) 的簡易 8 位元中央處理器 (Simple CPU)。該處理器內部整合了一組米利型 (Mealy Machine) 架構的有限狀態機 (FSM) 作為硬體控制單元，並配置有 4 個 8 位元通用暫存器 (R0~R3) 來進行資料存取與內部匯流排 (Bus) 傳輸。

本處理器完全支援 16 位元的硬體指令輸入格式 (對應開發板上的指撥開關 SW[15:0])，其欄位配置與操作碼規格如下：

1. 指令格式配置：

- SW[7:0]：8 位元立即值資料欄位 (Data)。
- SW[11:8]：4 位元操作碼欄位 (Opcode)。
- SW[13:12]：2 位元目的暫存器選擇欄位 (Rs_sel)。
- SW[15:14]：2 位元來源暫存器選擇欄位 (Rt_sel)。

2. 硬體指令集 (ISA) 規格：

- **LOAD Rs (Opcode = 0000)**：將暫存器控制鏈指向指定目的暫存器，並將指撥開關輸入的 8 位元立即值鎖存至 Rs 當中 (Rs←Data)。
- **MOVE Rs, Rt (Opcode = 0001)**：驅動內部匯流排多工器，將來源暫存器 Rt 的資料暫存值透過 Bus 傳輸同步鎖存至目的暫存器 Rs 當中 (Rs←Rt)。

二、硬體週邊配置與腳位對照表

埠口名稱 (Port)	資料方向	位元寬度	對應板載硬體週邊 (Component)	FPGA 晶片腳位 (Pin)
KEY[0]	Input	1	觸發按鍵 KEY0 (手動觸發中央處理器系	PIN_M23
SW[7..0]	Input	8	指撥開關 SW7 ~ SW0 (8位元平行立即值	PIN_AB26 至
SW[11..8]	Input	4	指撥開關 SW11 ~ SW8 (4位元微指令操作	PIN_AC25 等週邊
SW[13..12]	Input	2	指撥開關 SW13 ~ SW12 (目的暫存器 Rs 選擇)	依 CPU.qsf 物理綁定配置
SW[15..14]	Input	2	指撥開關 SW15 ~ SW14 (來源暫存器 Rt 選擇)	依 CPU.qsf 物理綁定配置
HEX1, HEX0	Output	14	七段顯示器 HEX1, HEX0 (即時鎖存顯示	依 .qsf 主體輸
HEX3, HEX2	Output	14	七段顯示器 HEX3, HEX2 (即時靜態顯示目的暫存器 Rs 之數值)	依 .qsf 主體輸出綁定
HEX5, HEX4	Output	14	七段顯示器 HEX5, HEX4 (即時靜態顯示來源暫存器 Rt 之數值)	依 .qsf 主體輸出綁定

三、實驗心得與深度思考

1. **搞懂處理器中控制路徑與資料路徑的並行本質：**在這次實作 Simple CPU 的過程中，我深刻體會到數位晶片設計中「控制路徑（Control Path）」與「資料路徑（Data Path）」的雙 Process 分工。控制路徑透過米利型狀態機動態偵測指撥開關的操作碼與暫存器選擇線，即時驅動資料路徑的多工器網路與通用暫存器陣列。這種全硬體並行的架構讓指令解碼與資料搬移能在同一個時脈週期內完成，這與軟體逐行執行的概念完全不同。
2. **同步復位與匯流排穩定設計的工程價值：**在分析 CPU.vhd 代碼時，我注意到所有暫存器的寫入控制都是同步鎖定在時脈訊號的正緣判斷內。這種設計可以完全過濾掉手動按鍵 KEY[0] 或切換指撥開關時產生的機械彈跳與毛邊訊訊號（Glitch），確保處理器內部的暫存器資料不會發生誤寫入。此外，將內部數值同時倒向七段顯示器即時輸出，也大幅降低了除錯的難度。
3. **簡易架構向複雜 ISA 擴充的工程實踐：**雖然這次實驗我們只實作了最基礎的 LOAD 與 MOVE 指令，但整個 CPU 的骨架已經非常完整。如果後續要進一步擴充暫存器數量、或是加入運算元（如結合 Lab 03 的 ALU 進行加減法運算），只需要在控制鏈的 Case 敘述中增加操作碼解碼條件並引入運算單元即可。這種高度模組化與可擴充性的架構設計，正是現代微處理器與計算機組織設計最核心的根本基石。

四、完整VHDL原始程式碼

```
-- =====
-- Lab 08: Simple CPU Implementation
-- Description:
--   A simple processor supporting LOAD and MOVE instructions.
--   Uses FSM (Mealy machine) with 4 general-purpose 8-bit registers (R0~R3).
--
-- Instruction Format (16-bit switch input):
-- SW[7:0]  = Data (8-bit immediate value)
-- SW[11:8] = Opcode (4-bit)
--           0000 = LOAD Rs    : Rs <- Data
--           0001 = MOVE Rs, Rt : Rs <- Rt
-- SW[13:12] = Rs (destination register select, 2-bit)
-- SW[15:14] = Rt (source register select, 2-bit)
--
-- Inputs:
-- SW[15:0] = Switches
-- KEY[0]   = Clock (push-button, active-low)
--
-- Outputs (7-segment displays, active-low):
-- HEX0, HEX1 = Bus value (8-bit, displayed as 2-digit hex)
```

```

-- HEX2, HEX3 = Rs register value
-- HEX4, HEX5 = Rt register value
-- =====

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity CPU is
  port (
    SW : in std_logic_vector(15 downto 0);
    KEY : in std_logic_vector(0 downto 0);
    HEX0 : out std_logic_vector(6 downto 0);
    HEX1 : out std_logic_vector(6 downto 0);
    HEX2 : out std_logic_vector(6 downto 0);
    HEX3 : out std_logic_vector(6 downto 0);
    HEX4 : out std_logic_vector(6 downto 0);
    HEX5 : out std_logic_vector(6 downto 0)
  );
end entity CPU;

architecture Behavioral of CPU is

  -- =====
  -- 7-Segment Decoder Function (active-low, common-anode style)
  -- Segments: gfedcba (bit 6 = g, bit 0 = a)
  -- =====
  function hex_to_7seg(hex_val : std_logic_vector(3 downto 0))
    return std_logic_vector is
    variable seg : std_logic_vector(6 downto 0);
  begin
    case hex_val is
      when "0000" => seg := "1000000"; -- 0
      when "0001" => seg := "1111001"; -- 1
      when "0010" => seg := "0100100"; -- 2
      when "0011" => seg := "0110000"; -- 3
      when "0100" => seg := "0011001"; -- 4
      when "0101" => seg := "0010010"; -- 5
      when "0110" => seg := "0000010"; -- 6
      when "0111" => seg := "1111000"; -- 7
      when "1000" => seg := "0000000"; -- 8
      when "1001" => seg := "0010000"; -- 9
      when "1010" => seg := "0001000"; -- A
      when "1011" => seg := "0000011"; -- b
      when "1100" => seg := "1000110"; -- C
      when "1101" => seg := "0100001"; -- d
      when "1110" => seg := "0000110"; -- E
      when "1111" => seg := "0001110"; -- F
      when others => seg := "1111111"; -- blank
    end case;
    return seg;
  end function hex_to_7seg;

  -- =====
  -- Internal signals
  -- =====

  -- Register file: R0, R1, R2, R3 (8-bit each)
  signal R0 : std_logic_vector(7 downto 0) := (others => '0');
  signal R1 : std_logic_vector(7 downto 0) := (others => '0');
  signal R2 : std_logic_vector(7 downto 0) := (others => '0');
  signal R3 : std_logic_vector(7 downto 0) := (others => '0');

```

```

-- Instruction decode
signal Data : std_logic_vector(7 downto 0); -- SW[7:0]
signal Opcode : std_logic_vector(3 downto 0); -- SW[11:8]
signal Rs_sel : std_logic_vector(1 downto 0); -- SW[13:12]
signal Rt_sel : std_logic_vector(1 downto 0); -- SW[15:14]

-- Clock: KEY[0] is active-low push button; rising edge = button release
-- We use the inverted signal so that pressing KEY[0] gives a rising edge
signal clk : std_logic;

-- Bus: 8-bit internal bus
signal Bus_val : std_logic_vector(7 downto 0);

-- Rs and Rt register values (combinational read)
signal Rs_val : std_logic_vector(7 downto 0);
signal Rt_val : std_logic_vector(7 downto 0);

-- Opcodes
constant OP_LOAD : std_logic_vector(3 downto 0) := "0000";
constant OP_MOVE : std_logic_vector(3 downto 0) := "0001";

begin

-- =====
-- Input decode
-- =====
Data <= SW(7 downto 0);
Opcode <= SW(11 downto 8);
Rs_sel <= SW(13 downto 12);
Rt_sel <= SW(15 downto 14);

-- Clock: KEY[0] is active-low; invert so rising edge = button press
clk <= not KEY(0);

-- =====
-- Combinational: Read Rs and Rt from register file
-- =====
process(Rs_sel, Rt_sel, R0, R1, R2, R3)
begin
-- Read Rs
case Rs_sel is
when "00" => Rs_val <= R0;
when "01" => Rs_val <= R1;
when "10" => Rs_val <= R2;
when others => Rs_val <= R3;
end case;

-- Read Rt
case Rt_sel is
when "00" => Rt_val <= R0;
when "01" => Rt_val <= R1;
when "10" => Rt_val <= R2;
when others => Rt_val <= R3;
end case;
end process;

-- =====
-- Combinational: Bus value
-- LOAD: Bus shows Data from SW[7:0]
-- MOVE: Bus shows Rt register value
-- Others: Bus shows 0x00
-- =====
process(Opcode, Data, Rt_val)

```

```

begin
  case Opcode is
    when OP_LOAD => Bus_val <= Data;
    when OP_MOVE => Bus_val <= Rt_val;
    when others => Bus_val <= (others => '0');
  end case;
end process;

-- =====
-- Sequential: Execute instruction on rising edge of clock (KEY[0] press)
-- =====
process(clk)
begin
  if rising_edge(clk) then
    case Opcode is

      -- LOAD Rs: Rs <- Data (SW[7:0])
      when OP_LOAD =>
        case Rs_sel is
          when "00" => R0 <= Data;
          when "01" => R1 <= Data;
          when "10" => R2 <= Data;
          when others => R3 <= Data;
        end case;

      -- MOVE Rs, Rt: Rs <- Rt
      when OP_MOVE =>
        case Rs_sel is
          when "00" => R0 <= Rt_val;
          when "01" => R1 <= Rt_val;
          when "10" => R2 <= Rt_val;
          when others => R3 <= Rt_val;
        end case;

      when others => null;

    end case;
  end if;
end process;

-- =====
-- Output: 7-Segment displays
-- HEX0, HEX1 = Bus value (lower nibble on HEX0, upper nibble on HEX1)
-- HEX2, HEX3 = Rs register value
-- HEX4, HEX5 = Rt register value
-- =====
HEX0 <= hex_to_7seg(Bus_val(3 downto 0));
HEX1 <= hex_to_7seg(Bus_val(7 downto 4));
HEX2 <= hex_to_7seg(Rs_val(3 downto 0));
HEX3 <= hex_to_7seg(Rs_val(7 downto 4));
HEX4 <= hex_to_7seg(Rt_val(3 downto 0));
HEX5 <= hex_to_7seg(Rt_val(7 downto 4));

end architecture Behavioral;

```

五、組員貢獻比例及工作內容

組員姓名	貢獻比例	具體負責工作內容描述
楊承諭	65%	小組專案製作與測試
黃楷程	35%	小組報告製作